

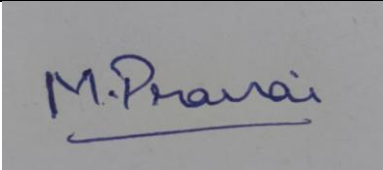
SIMATS ENGINEERING
CO4 - ASSESSMENT TOOL 2
Industry Problem Solving Task

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

TOTAL MARKS: 25

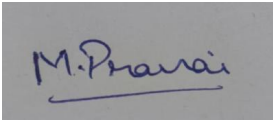
ASSESSMENT TOOL WEIGHTAGE: 40%

Name	M.Prnai
Register Number	192525068
Digital Signature	

Student Certification

I M.Prnai (192525068) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the student:



COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. (BL-6)

Note: The assignment questions below are reproduced in the same order and terminology as the uploaded question paper.

Q1. Transfer Learning

A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Q2. DenseNet and PixelNet

A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

Q3. Bidirectional RNN and Encoder–Decoder

A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

Q4. BPTT and LSTM

A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Q5. Integrated Deep Learning Solution

A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Total rubric weightage: 95% as listed in the supplied question paper. The remaining assessment weightage, if applicable, should be interpreted according to faculty/course instructions.

ANSWERS

Q1. Transfer Learning

1. Problem Understanding

The startup has a binary or multi-class image-classification problem in which dermoscopic images must be assigned to clinically relevant skin-lesion categories. The major constraint is the small number of labeled training images. Training a deep CNN completely from random initialization would require much more labeled data and could overfit.

2. Proposed Transfer Learning Strategy

Use a pre-trained EfficientNet-B0 or ResNet-50 CNN trained on a large natural-image dataset as the feature extractor. EfficientNet-B0 is a practical choice because it provides strong image features with relatively low computational cost. The final classification layer is replaced with a new task-specific head for the skin-lesion classes.

3. Architecture

Input dermoscopic image → resize/normalization → pre-trained CNN backbone → global average pooling → dropout → fully connected layer → softmax/sigmoid output → predicted lesion class.

4. Layer Freezing and Fine-Tuning

During the first training stage, freeze the convolutional backbone and train only the new classification head. This preserves general visual features such as edges, textures, shapes and colour patterns while adapting the final representation to the medical task.

After the classification head has converged, unfreeze the last few convolutional blocks and fine-tune them using a very small learning rate. Earlier layers should remain frozen because their generic features are useful and changing them with limited medical data can increase overfitting. The final classification layer and the later high-level feature blocks receive the greatest task-specific adaptation.

5. Data Preparation and Augmentation

Images should be resized to the CNN input size and normalized consistently. Training augmentation can include small rotations, horizontal/vertical flips where clinically appropriate, random crops, mild zooming and brightness/contrast variation. Augmentation should not create unrealistic lesions. A patient-level train/validation/test split is important so that images from the same patient do not leak across partitions.

6. Training Procedure

Stage 1: freeze the backbone and train the new classifier using Adam or another suitable optimizer.

Stage 2: unfreeze the last few backbone blocks and fine-tune with a lower learning rate.

Use early stopping, learning-rate reduction and model checkpointing. If class imbalance exists, use class-weighted loss or a suitable sampling strategy. Performance should be evaluated with sensitivity, specificity, precision, recall, F1-score and ROC-AUC rather than accuracy alone.

7. Why the Design Improves Classification Performance

Transfer learning starts from useful visual representations learned from a large dataset. Therefore, the model needs fewer medical examples to learn meaningful features. Freezing early layers reduces the number of trainable parameters and limits overfitting. Fine-tuning the later layers allows the network to adapt high-level features to dermoscopic patterns. Data augmentation improves robustness, while patient-level splitting and independent testing make the reported performance more reliable.

8. Industry Feasibility and Conclusion

The proposed system is practical for a startup because it reduces training time, data requirements and computational cost compared with training a deep CNN from scratch. The model should be treated as a decision-support system and validated on clinically representative external data before deployment. Overall, a pre-trained CNN with a new classification head followed by controlled fine-tuning provides an effective transfer-learning solution for limited labeled dermoscopic data.

Q2. DenseNet and PixelNet

1. Problem Understanding

The requirement is semantic segmentation: every pixel in a road-scene image must be assigned to a class such as road, pedestrian, vehicle, traffic sign or background. This is more demanding than image classification because the system must preserve spatial information and produce a dense pixel-level output.

2. Proposed Architecture

Use DenseNet as the feature-extraction backbone and a PixelNet-style pixel-level prediction component as the dense segmentation head. The overall flow is:

Input road image → preprocessing → DenseNet feature extractor → multi-scale feature representations → feature fusion/upsampling → PixelNet pixel-wise prediction layers → per-pixel class probabilities → segmentation mask.

3. Role of DenseNet

DenseNet connects each layer to subsequent layers within a dense block. Earlier feature maps are therefore reused by later layers. This encourages feature reuse, improves gradient flow and reduces unnecessary relearning of similar features. For road scenes, low-level edges and textures can be combined with deeper semantic information to distinguish objects such as pedestrians, vehicles and signs.

4. Role of PixelNet

The PixelNet component performs pixel-level prediction rather than producing only one class for the whole image. Dense feature maps from the backbone are transformed into per-pixel class scores. A softmax layer can assign each pixel to the most probable semantic category.

5. Multi-Scale Segmentation Design

Road scenes contain objects at very different sizes. The network should therefore combine shallow high-resolution features with deeper semantic features. Skip connections can bring spatial detail from earlier DenseNet stages to the decoder. Upsampling gradually restores the original image resolution, while fused features improve object boundaries and small-object recognition.

6. Feature Reuse and Segmentation Accuracy

Dense connections allow the network to reuse previously learned features. This improves information flow and helps the model learn discriminative representations with fewer redundant transformations. Combining high-resolution shallow features and semantic deep features improves segmentation boundaries, particularly for pedestrians, vehicles and traffic signs that occupy relatively small image regions.

7. Computational Efficiency

DenseNet can achieve strong feature extraction with parameter-efficient connectivity and feature reuse. A compact DenseNet backbone can reduce computation compared with a very large backbone. The segmentation head can use lightweight convolutional layers and progressive upsampling. Input resolution, batch size and the number of decoder channels can also be selected according to the available hardware.

8. Training and Evaluation

Train the network using pixel-wise cross-entropy or a combined loss such as cross-entropy plus Dice loss when class imbalance is significant. Evaluate using mean Intersection over Union (mIoU), per-class IoU, pixel accuracy

and boundary quality. A validation set should be used for model selection and an independent test set for final evaluation.

9. Conclusion

The DenseNet–PixelNet design combines efficient feature reuse with dense pixel-level prediction. DenseNet supplies rich multi-level features, while the PixelNet-style head converts those features into semantic segmentation masks. The combination is suitable for smart-city road-scene analysis because it can identify road users and traffic infrastructure at pixel level while controlling computational cost.

Q3. Bidirectional RNN and Encoder–Decoder

1. Problem Understanding

The system receives a customer query in one language and generates an equivalent response or translated sequence in another language. Word order and context matter, so a sequence-to-sequence architecture is appropriate. Multilingual processing also requires a shared representation capable of capturing language-independent meaning.

2. Proposed Architecture

Source sentence → tokenization → embeddings → Bidirectional RNN encoder → context representation → decoder RNN → attention/context information → target token prediction → generated translated sentence.

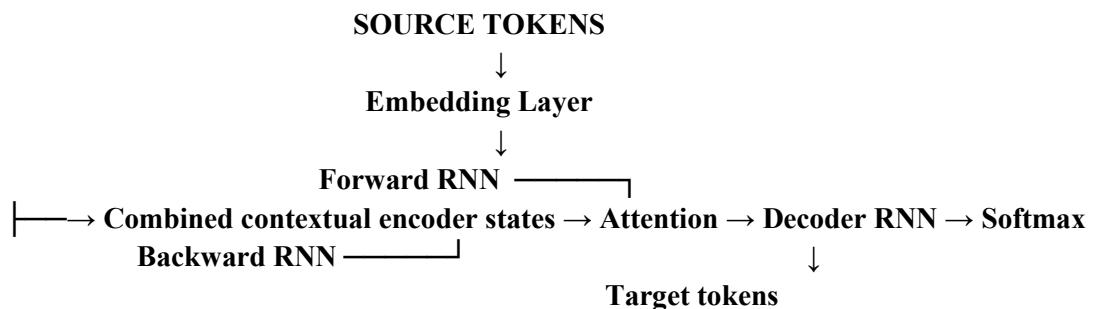
3. Bidirectional Encoder

A Bidirectional RNN processes the input sequence in both directions. One recurrent layer reads the sequence from the first token to the last, while another reads it from the last token to the first. Their hidden representations are combined. Consequently, the representation of a token can incorporate information from both its preceding and following context.

4. Encoder–Decoder Operation

The encoder converts the source sequence into contextual hidden representations. The decoder generates the target sequence one token at a time. At each step it uses the previous generated token, its recurrent state and the relevant encoder information. An attention mechanism is recommended so the decoder can focus on different source words as it generates different target words.

5. Architecture Illustration



6. Why Bidirectional Processing Improves Accuracy

A conventional one-directional encoder mainly builds context from one side of the sentence. A bidirectional encoder uses both past and future context. This is useful when the meaning of a word depends on words that occur later in the sentence. The resulting contextual representation can reduce ambiguity and improve translation of phrases where local word meaning changes with surrounding words.

7. Multilingual Design

The system can use language identifiers or special language tokens so the decoder knows the required target language. Shared embeddings and a common encoder can learn reusable semantic patterns across languages. Training should include balanced or carefully sampled multilingual data to avoid severe performance differences between high-resource and low-resource languages.

8. Training and Evaluation

Train with teacher forcing and cross-entropy loss. Use padding masks for variable-length sequences and an end-of-sequence token to terminate generation. Evaluate with BLEU or other translation metrics and, importantly, human evaluation for fluency, adequacy and preservation of customer intent. For deployment, latency and memory usage should also be monitored.

9. Conclusion

The Bidirectional RNN Encoder–Decoder architecture provides a context-aware sequence-to-sequence solution. Bidirectional encoding captures information from both sides of the source sentence, while the decoder generates the target sequence. Attention further connects relevant source information to each generated word, improving contextual translation quality for customer-service conversations.

Q4. BPTT and LSTM

1. Problem Understanding

Stock forecasting is a time-series prediction problem. Historical prices and related variables form a sequence, and the model must use information from previous time steps to predict a future value or direction. Financial data are noisy and may contain dependencies over different time scales.

2. Proposed LSTM System

Historical market window → feature normalization → LSTM layers → dropout → dense output layer → predicted next price/return/direction.

3. Input Features

Possible inputs include open, high, low, close and volume values. Additional engineered variables such as returns or moving averages may be used if they are available without future-data leakage. The data should be ordered chronologically and split into training, validation and test periods rather than randomly shuffled across time.

4. LSTM Cell

An LSTM maintains a cell state and uses gates to control information flow:

Forget	gate:	decides	which	old	information	to	remove.
Input	gate:	decides	which	new	information	to	store.
Candidate		state:	creates		candidate		information.
Output	gate:	controls	what	information	is	exposed	as the hidden state.

These mechanisms allow the network to preserve useful information for longer periods and reduce the vanishing-gradient problem that affects ordinary RNNs.

5. Backpropagation Through Time (BPTT)

During training, the LSTM is conceptually unrolled across the selected historical time steps. The model produces predictions, the prediction error is calculated using a loss function such as mean squared error for regression, and gradients are propagated backward through the unrolled sequence. The shared LSTM parameters are updated using these gradients.

For a long sequence, full BPTT can be expensive. Truncated BPTT can instead backpropagate through a fixed number of time steps, reducing memory and computational requirements.

6. Why LSTM Is More Suitable Than Traditional RNN

Traditional RNNs can struggle to preserve information over long sequences because repeated multiplication through time can cause vanishing or exploding gradients. LSTM's gated cell-state mechanism provides a controlled path for information and gradients. Therefore, it is better suited when useful dependencies may extend over many historical observations.

7. Training and Validation

Normalize training features using statistics calculated only from the training period. Use a rolling or expanding validation strategy when appropriate. Monitor validation loss, use early stopping and avoid excessive model complexity. For regression, evaluate MAE and RMSE; for directional prediction, use accuracy, precision, recall or F1-score as appropriate.

8. Important Financial Considerations

A realistic system must avoid look-ahead bias and data leakage. Transaction costs, slippage and changing market conditions should be considered if predictions are converted into trading decisions. A low prediction error does not automatically imply a profitable strategy, so financial performance must be evaluated separately.

9. Conclusion

The proposed LSTM system learns temporal patterns from historical market sequences. BPTT trains the recurrent network by propagating prediction errors through time, while LSTM gates help preserve useful long-term information. This makes LSTM a more robust choice than a basic RNN for sequence forecasting, although financial predictions remain uncertain and require rigorous out-of-sample validation.

Q5. Integrated Deep Learning Solution

1. Problem Understanding

The system must convert visual information from a video into a natural-language caption sequence. The video contains multiple frames and temporal events, so the solution must extract useful visual representations and then generate a grammatically meaningful sequence of words.

2. Integrated Architecture

Video → frame sampling → pre-trained CNN/transfer-learning feature extractor → frame-level visual embeddings → temporal aggregation/sequence representation → LSTM Encoder → context representation → LSTM Decoder → word probabilities → generated caption.

3. Transfer Learning Component

A CNN pre-trained on a large image dataset is used to extract visual features from sampled video frames. The final classification layer is removed or replaced so that the network functions as a feature extractor. Initially, most CNN layers are frozen. If sufficient task-specific data are available, selected later layers can be fine-tuned using a small learning rate.

4. Visual Feature Extraction

Instead of training a CNN from scratch on the captioning dataset, each selected video frame is converted into a compact feature vector. Frame sampling reduces redundant computation. The resulting sequence of feature vectors represents how the visual content changes over time.

5. LSTM Encoder

The LSTM encoder reads the sequence of visual feature vectors and maintains temporal information. It can learn that a sequence of frames represents an event rather than treating every frame independently. The final hidden representation or attention-based sequence representation is passed to the decoder.

6. LSTM Decoder

The decoder generates the caption one token at a time. It starts with a start-of-sequence token and predicts the next word using its hidden state and visual context. The predicted word is fed back into the decoder until an end-of-sequence token is produced or a maximum length is reached.

7. Attention and Accuracy Improvement

An attention mechanism can allow the decoder to focus on the most relevant video-frame features for each generated word. For example, the decoder can attend more strongly to frames containing an action while generating the action word. This reduces the burden of compressing the entire video into one fixed vector and can improve caption relevance.

8. Training Strategy

Train the captioning model using paired video-caption data. Tokenize captions, create a vocabulary and use padding masks for variable-length sequences. Teacher forcing can be used during training. Cross-entropy loss measures the difference between predicted and ground-truth tokens. Scheduled sampling or other sequence-training strategies may be considered carefully for advanced implementations.

9. Evaluation

Caption quality can be measured using BLEU, ROUGE, METEOR, CIDEr or similar automatic metrics, but human evaluation is also valuable for relevance, fluency and factual correctness. The system should be tested on unseen videos and across different scenes, speakers, environments and accessibility requirements.

10. Complete Workflow and Conclusion

The integrated pipeline combines the strengths of two components. Transfer learning provides strong visual representations without requiring a huge labelled video dataset. The LSTM Encoder–Decoder converts the visual sequence into language while learning temporal relationships. Attention links generated words to relevant visual content. Together, these components provide a practical architecture for automated video caption generation and can improve accessibility when validated on representative real-world content.

PROGRAMS / PYTHON IMPLEMENTATION

The following programs provide practical Python/Keras implementations corresponding to Q1–Q5. They are suitable for demonstration and GitHub submission. Dataset paths and training parameters can be adjusted according to the available dataset and hardware.

PROGRAM 1 – Transfer Learning for Skin Cancer Classification (Q1)

Implementation note: replace the example dataset paths/files with the actual datasets used for the project.

```
# Q1: Transfer Learning using EfficientNetB0
# Example for binary/multi-class dermoscopic image classification
```

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import EfficientNetB0

IMG_SIZE = (224, 224)
BATCH_SIZE = 16
NUM_CLASSES = 2

train_ds = tf.keras.utils.image_dataset_from_directory(
    "dataset/train",
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int"
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    "dataset/validation",
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int"
)

data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.10),
    layers.RandomZoom(0.10),
    layers.RandomContrast(0.10)
])

base_model = EfficientNetB0(
    include_top=False,
    weights="imagenet",
    input_shape=IMG_SIZE + (3,)
)

# Stage 1: freeze the pre-trained backbone
base_model.trainable = False

inputs = layers.Input(shape=IMG_SIZE + (3,))
x = data_augmentation(inputs)
x = tf.keras.applications.efficientnet.preprocess_input(x)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.30)(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

model = models.Model(inputs, outputs)

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```

model.fit(train_ds, validation_data=val_ds, epochs=10)

# Stage 2: fine-tune the last part of the backbone
base_model.trainable = True

for layer in base_model.layers[:-30]:
    layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.fit(train_ds, validation_data=val_ds, epochs=10)

model.save("skin_cancer_transfer_learning.keras")
print("Model saved successfully.")

```

PROGRAM 2 – DenseNet + Pixel-Level Road Scene Segmentation (Q2)

Implementation note: replace the example dataset paths/files with the actual datasets used for the project.

```
# Q2: DenseNet backbone with a pixel-level segmentation head
# Classes: 0=background, 1=road, 2=pedestrian,
#          3=vehicle, 4=traffic_sign

import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.applications import DenseNet121

IMG_SIZE = (256, 256)
NUM_CLASSES = 5

backbone = DenseNet121(
    include_top=False,
    weights="imagenet",
    input_shape=IMG_SIZE + (3,)
)

backbone.trainable = False

inputs = layers.Input(shape=IMG_SIZE + (3,))
x = tf.keras.applications.densenet.preprocess_input(inputs)

features = backbone(x, training=False)

# Lightweight pixel-level decoder
x = layers.Conv2D(256, 3, padding="same", activation="relu")(features)
x = layers.UpSampling2D((2, 2))(x)
x = layers.Conv2D(128, 3, padding="same", activation="relu")(x)
x = layers.UpSampling2D((2, 2))(x)
x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
x = layers.UpSampling2D((2, 2))(x)
x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)

# Restore the required image resolution
x = layers.Resizing(IMG_SIZE[0], IMG_SIZE[1])(x)

outputs = layers.Conv2D(
    NUM_CLASSES, 1, activation="softmax", name="pixel_predictions"
)(x)

model = Model(inputs, outputs)

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-4),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# mask_dataset must contain integer masks with shape (256,256,1)
# Example:
# model.fit(image_mask_dataset, epochs=20, validation_data=val_dataset)

model.summary()
print("DenseNet-PixelNet style segmentation model created.")
```


PROGRAM 3 – Bidirectional RNN Encoder–Decoder for Translation (Q3)

Implementation note: replace the example dataset paths/files with the actual datasets used for the project.

```
# Q3: Bidirectional RNN Encoder-Decoder
# Simplified sequence-to-sequence translation model

import tensorflow as tf
from tensorflow.keras import layers, Model

SRC_VOCAB = 10000
TGT_VOCAB = 10000
EMBED_DIM = 256
UNITS = 256
MAX_LEN = 30

# Source input: token IDs of the customer query
encoder_inputs = layers.Input(
    shape=(MAX_LEN,), dtype="int32", name="source_sentence"
)

encoder_embedding = layers.Embedding(
    SRC_VOCAB, EMBED_DIM, mask_zero=True
)(encoder_inputs)

# Bidirectional encoder
encoder_output = layers.Bidirectional(
    layers.GRU(UNITS, return_sequences=True)
)(encoder_embedding)

# Decoder input contains target tokens shifted by one position
decoder_inputs = layers.Input(
    shape=(MAX_LEN,), dtype="int32", name="target_sentence"
)

decoder_embedding = layers.Embedding(
    TGT_VOCAB, EMBED_DIM, mask_zero=True
)(decoder_inputs)

decoder_rnn = layers.GRU(
    UNITS * 2, return_sequences=True
)(decoder_embedding)

# Attention between decoder states and bidirectional encoder states
attention = layers.AdditiveAttention()(
    [decoder_rnn, encoder_output]
)

combined = layers.Concatenate()(
    [decoder_rnn, attention]
)

outputs = layers.Dense(
    TGT_VOCAB, activation="softmax"
)(combined)

model = Model(
    [encoder_inputs, decoder_inputs],
    outputs
)

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# Training format:
```

```
# model.fit(  
#     [source_train, target_input_train],  
#     target_output_train,  
#     validation_data=(  
#         [source_val, target_input_val],  
#         target_output_val  
#     ),  
#     epochs=20  
# )  
  
print("Bidirectional RNN Encoder-Decoder model created.")
```

PROGRAM 4 – LSTM Stock Price Prediction using BPTT (Q4)

Implementation note: replace the example dataset paths/files with the actual datasets used for the project.

```
# Q4: LSTM-based time-series prediction
# BPTT is performed automatically by Keras during model.fit()

import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

# CSV should contain a 'Close' column
data = pd.read_csv("stock_data.csv")
prices = data[["Close"]].values.astype("float32")

scaler = MinMaxScaler()
scaled_prices = scaler.fit_transform(prices)

WINDOW = 30
X, y = [], []

for i in range(WINDOW, len(scaled_prices)):
    X.append(scaled_prices[i-WINDOW:i])
    y.append(scaled_prices[i])

X = np.array(X)
y = np.array(y)

# Chronological split - do not randomly shuffle time-series data
split = int(0.8 * len(X))

X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(WINDOW, 1)),
    Dropout(0.2),
    LSTM(32),
    Dropout(0.2),
    Dense(1)
])

model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

# During training, Keras performs Backpropagation Through Time
# through the unrolled LSTM sequence.
history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.1,
    shuffle=False
)

loss, mae = model.evaluate(X_test, y_test)
print("Test MSE:", loss)
print("Test MAE:", mae)

predicted = model.predict(X_test)
predicted_prices = scaler.inverse_transform(predicted)

print("First predicted price:", predicted_prices[0][0])
```


PROGRAM 5 – Transfer Learning + LSTM Encoder–Decoder Video Captioning (Q5)

Implementation note: replace the example dataset paths/files with the actual datasets used for the project.

```
# Q5: Video Captioning
# Transfer learning extracts frame features; LSTM generates captions.
# This is a compact demonstration architecture.
```

```
import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.applications import MobileNetV2

IMG_SIZE = (224, 224)
FEATURE_DIM = 1280
VOCAB_SIZE = 10000
EMBED_DIM = 256
UNITS = 256
MAX_LEN = 30

# -----
# 1. Transfer-learning CNN
# -----
cnn = MobileNetV2(
    include_top=False,
    weights="imagenet",
    pooling="avg",
    input_shape=IMG_SIZE + (3,)
)

cnn.trainable = False

frame_input = layers.Input(
    shape=IMG_SIZE + (3,),
    name="video_frame"
)

x = tf.keras.applications.mobilenet_v2.preprocess_input(
    frame_input
)

frame_features = cnn(x, training=False)

feature_extractor = Model(
    frame_input,
    frame_features
)

# A video becomes a sequence of frame feature vectors:
# [feature_1, feature_2, ..., feature_T]

# -----
# 2. LSTM Encoder
# -----
video_features = layers.Input(
    shape=(None, FEATURE_DIM),
    name="video_features"
)

encoder = layers.LSTM(
    UNITS,
    return_sequences=True,
    return_state=True
)

encoder_outputs, state_h, state_c = encoder(video_features)

# -----
# 3. LSTM Decoder
# -----
caption_inputs = layers.Input(
    shape=(MAX_LEN,),
```

```

        dtype="int32",
        name="caption_input"
    )

    embedding = layers.Embedding(
        VOCAB_SIZE, EMBED_DIM, mask_zero=True
    )(caption_inputs)

    decoder = layers.LSTM(
        UNITS,
        return_sequences=True
    )

    decoder_outputs = decoder(
        embedding,
        initial_state=[state_h, state_c]
    )

    # -----
    # 4. Word prediction
    # -----
    word_outputs = layers.Dense(
        VOCAB_SIZE,
        activation="softmax"
    )(decoder_outputs)

    caption_model = Model(
        [video_features, caption_inputs],
        word_outputs
    )

    caption_model.compile(
        optimizer="adam",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    caption_model.summary()

    # Training example:
    # caption_model.fit(
    #     [video_feature_train, caption_input_train],
    #     caption_output_train,
    #     validation_data=(
    #         [video_feature_val, caption_input_val],
    #         caption_output_val
    #     ),
    #     epochs=20
    # )

    print("Video captioning Encoder-Decoder model created.")

```

GITHUB SUBMISSION NOTE

For the faculty instruction to upload programs in the GitHub CO4 - AT2 folder, the five programs above can be saved as separate Python files: Q1_transfer_learning.py, Q2_densenet_pixlnet.py, Q3_birnn_encoder_decoder.py, Q4_lstm_bptt_stock.py, and Q5_video_captioning.py. The code in this document is a demonstration implementation; actual dataset loading, annotation preparation and training parameters should be updated for the datasets used.